

Classes Abstratas, Frameworks

Eduardo Piveta

Reuso de Software

- Produção e disponibilização de artefatos de software reutilizáveis
- Tentativas
 - Princípios de modularização
 - Reuso de Classes:
 - Composição
 - Herança

Granularidade

Rotinas Classes Componentes Padrões Frameworks



-

+

Frameworks

- Frameworks são uma técnica de reuso de software OO
- Definição:
 - Projeto reutilizável de parte de um sistema, representado por um conjunto de classes abstratas e seu inter-relacionamento (estrutura)
 - Esqueleto de uma aplicação que pode ser customizado pelo desenvolvedor (propósito)

Classes Abstratas em Java

- Classes abstratas são classes que NÃO podem ser instanciadas...
- ...somente suas sub-classes podem ser instanciadas.
- Elas podem ter:
 - Métodos concretos
 - Métodos abstratos
 - Estes são sobrescritos nas sub-classes

Classes com Tipos Genéricos

- Possuem tipos que só são definidos na declaração de variáveis, na instanciação de objetos ou pelas sub-classes.
- Vocês já viram como usar tipos genéricos ao usarem a interface Collection e a classe ArrayList, por exemplo.
- Agora veremos como criar tipos genéricos...

Exemplo: Acesso a Dados (DAOs)

- Criaremos uma classe abstrata e genérica para lidar com as operações de:
 - Inserção
 - Alteração
 - Exclusão
 - Busca
 - Conexão com o BD
- Não são realizados tratamentos de exceções (ainda)....

Classe: Registros

```
public abstract class Registros<T extends Registro> {  
    private String sqlInsercao;  
    private String sqlAlteracao;  
    private String sqlExclusao;  
    private String sqlBuscaChavePrimaria;  
    private String sqlBuscaTodos;
```

Baixar o exemplo DBTesteGenerico...

Registros: Abrir conexão

```
public Connection abrir() {  
    Connection c = null;  
    try {  
        Class.forName(BD.JDBC_DRIVER);  
    } catch (ClassNotFoundException e) {  
        System.out.println("Nao encontrou o driver chamado " + BD.JDBC_DRIVER + " na memoria");  
    }  
    try {  
        c = DriverManager.getConnection(BD.URL_CONEXAO, BD.USUARIO, BD.SENHA);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return c;  
}
```

Registros: Inclusão

```
public void inserir(Cliente c) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(  
            "INSERT INTO clientes (codigo, nome) VALUES (?, ?)");  
        ps.setInt(1, c.getCodigo());  
        ps.setString(2, c.getNome());  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```



```
public void inserir(T t) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(getSqlInsercao());  
        preencherInsercao(ps, t);  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Registros: Alteração

```
public void alterar(Cliente c) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(  
            "UPDATE clientes SET nome = ? WHERE codigo = ?");  
        ps.setString(1, c.getNome());  
        ps.setInt(2, c.getCodigo());  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```



```
public void alterar(T t) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(getSqlAlteracao());  
        preencherAlteracao(ps, t);  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Registros: Exclusão

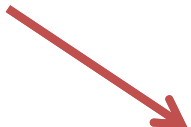
```
public void excluir(Cliente c) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(  
            "DELETE FROM clientes WHERE codigo = ?");  
        ps.setInt(1, c.getCodigo());  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```



```
public void excluir(T t) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(getSqlExclusao());  
        preencherExclusao(ps, t);  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Registros: Buscar Todos

```
public Collection<Cliente> buscarTodos() {  
    Connection conexao = abrir();  
    Collection<Cliente> clientes = new ArrayList<Cliente>();  
    try {  
        Statement s = conexao.createStatement();  
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");  
        while (rs.next()) {  
            Cliente temp = new Cliente();  
            temp.setCodigo(rs.getInt("codigo"));  
            temp.setNome(rs.getString("nome"));  
            clientes.add(temp);  
        }  
        rs.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return clientes;  
}
```



```
public Collection<T> buscarTodos() throws SQLException {  
    Connection conexao = abrir();  
    Statement s = conexao.createStatement();  
    ResultSet rs = s.executeQuery(getSqlBuscaTodos());  
    Collection<T> registros = preencherColecao(rs);  
    rs.close();  
    conexao.close();  
    return registros;  
}
```

Registros: Buscar Individualmente

```
public T buscar(T t) {  
    Connection conexao = abrir();  
    T temp = null;  
    try {  
        PreparedStatement ps = conexao.prepareStatement(getSqlBusca());  
        preencherExclusao(ps, t);  
        ResultSet rs = ps.executeQuery();  
        temp = preencher(rs);  
        rs.close();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return temp;  
}
```

Registros: Delegando a Implementação para as sub-classes

```
protected abstract void preencherInsercao(PreparedStatement ps, T t) throws SQLException;

protected abstract void preencherAlteracao(PreparedStatement ps, T t) throws SQLException;

protected abstract void preencherExclusao(PreparedStatement ps, T t) throws SQLException;

protected abstract T preencher(ResultSet rs) throws SQLException;

protected abstract Collection<T> preencherColecao(ResultSet rs) throws SQLException;
```

ClientesDAO

```
public class ClientesDAO extends Registros<Cliente> {  
    public ClientesDAO() {  
        setSqlInsercao("INSERT INTO clientes (codigo, nome) VALUES (?, ?)");  
        setSqlAlteracao("UPDATE clientes SET nome = ? WHERE codigo = ?");  
        setSqlExclusao("DELETE FROM clientes WHERE codigo = ?");  
        setSqlBuscaChavePrimaria("SELECT * FROM clientes WHERE codigo = ?");  
        setSqlBuscaTodos("SELECT * FROM clientes");  
    }  
  
    @Override  
    protected void preencherInsercao(PreparedStatement ps, Cliente c) throws SQLException {  
        ps.setInt(1, c.getCodigo());  
        ps.setString(2, c.getNome());  
    }  
  
    @Override  
    protected void preencherAlteracao(PreparedStatement ps, Cliente c) throws SQLException {  
        ps.setString(1, c.getNome());  
        ps.setInt(2, c.getCodigo());  
    }  
  
    @Override  
    protected void preencherExclusao(PreparedStatement ps, Cliente c) throws SQLException {  
        ps.setInt(1, c.getCodigo());  
    }  
}
```


ClientesDAO

```
@Override
protected Cliente preencher(ResultSet rs) throws SQLException {
    Cliente cliente = new Cliente();
    cliente.setCodigo(rs.getInt("codigo"));
    cliente.setNome(rs.getString("nome"));
    return cliente;
}

@Override
protected Collection<Cliente> preencherColecao(ResultSet rs) throws SQLException {
    Collection<Cliente> retorno = new ArrayList<Cliente>();
    while (rs.next()) {
        Cliente temp = new Cliente();
        temp.setCodigo(rs.getInt("codigo"));
        temp.setNome(rs.getString("nome"));
        retorno.add(temp);
    }
    return retorno;
}
```

Cliente

```
public class Cliente extends Registro{  
    private Integer codigo;  
    private String nome;
```

```
public abstract class Registro {  
  
}
```

Exercício

- Implementar as classes de exemplo:
 - DBExemploConexao
 - ClientesDAO
 - Cliente
 - BD
 - Registros
 - Registro
- Listar os registros da tabela após cada operação. Ex.:
 - Listar todos os clientes
 - Apagar todos os clientes
 - Listar todos os clientes novamente
 - Inserir clientes
 - Listar todos os clientes novamente
 - Alterar clientes
 - Listar todos os clientes novamente
- Comparar com DBTesteGenerico.zip do site...